Project Type: MEMORY VERIFICATION USING SYSTEMVERILOG

Author: Narasimhamurthy B

1. Overview

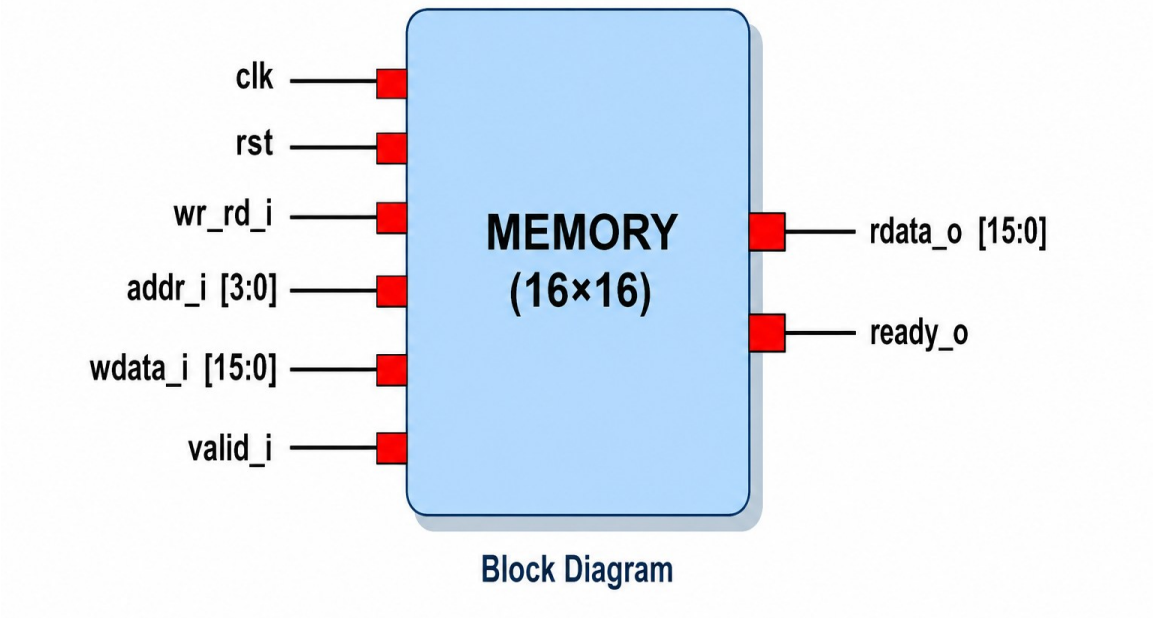
The `memory` module is a parameterized synchronous memory design verified using a self-checking SystemVerilog testbench environment.

The project implements:

- Memory RTL Design
- Constrained Random Verification
- Mailbox-Based Communication
- Driver
- Monitor
- Scoreboard
- Functional Coverage
- Cross Coverage
- Interface and Clocking Blocks

The verification environment validates both write and read operations using randomized transactions and automatic comparison inside the scoreboard.

2. Block Diagram



Block Diagram

3. Memory Module Description

The memory module is a parameterized synchronous memory.

Features

- Synchronous write operation
- Synchronous read operation
- Parameterized memory depth and width
- Ready/valid style handshake
- Reset support
- Addressable memory array

4. RTL Functionality

Write Operation

When:

- `valid_i = 1`
- `wr_rd_i = 1`

The input data (`wdata_i`) is written into memory at address `addr_i`.

Read Operation

When:

- `valid_i = 1`
- `wr_rd_i = 0`

The memory returns data from address `addr_i` through `rdata_o`.

Reset Behavior

When `rst = 1`:

- Memory contents are cleared
- `rdata_o` becomes 0
- `ready_o` becomes 0

5. Signal Description

Signal Name	Direction	Width	Description
clk	Input	1	System clock
rst	Input	1	Active-high reset
wr_rd_i	Input	1	Write/Read control signal
valid_i	Input	1	Valid transaction indicator
addr_i	Input	ADDR_WIDTH	Memory address input
wdata_i	Input	WIDTH	Write data input
rdata_o	Output	WIDTH	Read data output
ready_o	Output	1	Ready signal

6. Parameters

Parameter	Description
DEPTH	Number of memory locations
WIDTH	Width of memory data
ADDR_WIDTH	Address width calculated using `$clog2(DEPTH)`

Default configuration:

DEPTH = 16

WIDTH = 16

ADDR_WIDTH = 4

7. Verification Environment

The verification environment is fully self-checking.

Verification Components

7.1 Generator

The generator creates randomized transactions.

Features:

- Randomized write transactions
- Randomized read transactions
- Address matching between write and read
- Constrained random verification

7.2 Driver

The Driver drives randomized transactions to the DUT through the interface.

Functions:

- Drives write transactions
- Drives read transactions
- Handles handshake signals
- Synchronizes transactions with clocking blocks

7.3 Monitor

The monitor observes DUT activity through the interface.

Functions:

- Captures write transactions
- Captures read transactions
- Sends transactions to scoreboard and coverage
- Avoids duplicate transaction sampling

7.4 Scoreboard

The scoreboard acts as a reference model.

Functions:

- Stores expected memory values
- Compares DUT read data with expected data
- Displays PASS/FAIL messages automatically

Example output:

PASS addr_i = 6 rdata = 52730

7.5 Functional Coverage

Functional coverage is implemented using cover groups.

Coverage includes:

- Read operations
- Write operations
- Low address access
- Middle address access
- High address access
- Cross coverage between operation type and address range

Coverage model:

Cover group mem_cg;

Example simulation result:

Coverage = 100.00 %

8. Mailbox Communication

The project uses mailbox-based communication between components.

Mailbox Purpose

Grn2drv Generator to Driver

Mon2scb Monitor to Scoreboard

Mon2cov Monitor to Coverage

9. Interface and Clocking Blocks

The project uses a SystemVerilog interface for clean DUT connectivity.

Clocking blocks are used for:

- Proper synchronization
- Race condition avoidance
- Stable signal sampling

Clocking blocks used:

drv_cb

mon_cb

10. Verification Flow

Write Transaction Flow

Generator -> Driver -> DUT -> Monitor -> Scoreboard

Read Transaction Flow

Generator -> Driver -> DUT -> Monitor -> Scoreboard -> PASS/FAIL Check

11. Simulation Results

The verification environment successfully validates:

- Memory write operations
- Memory read operations
- Read-after-write behavior

- Randomized address accesses
- Multiple memory locations
- Functional coverage goals

Simulation Status:

PASS Transactions : Successful

Coverage : 100%

Compile Errors : 0

Simulation Errors : 0

12. Applications

This project can be used for learning:

- RTL Verification
- SystemVerilog Testbench Architecture
- Constrained Random Verification
- Functional Coverage
- Self-Checking Testbenches
- Mailbox Communication
- Verification Methodology Concepts

14. Conclusion

This project demonstrates a complete SystemVerilog memory verification environment with constrained random stimulus generation, automatic checking, and functional coverage collection.

The environment successfully verifies memory read and write functionality while achieving full functional coverage.

The project is suitable for beginner and intermediate Design Verification learning and demonstrates important verification concepts used in real DV environments.